

Academic Thesis for the Attainment of the Degree of
Master of Science
at the Technical University of Munich

An Algorithmic Framework for Very Large-Scale Capacitated Vehicle Routing Problems

Examiner: Prof. Dr. Stefan Minner
Chair of Logistics and Supply Chain Management
Technische Universität München

Supervisor: Till Krieger, M.Sc.

Degree Program: Management & Technology Master

Submitted by:
Lucca Altendeitering
Stolzestr. 21, 44789 Bochum
Tel.: +49 162 7630077
Matriculation Number: 3782307

Date of Submission: 30.09.2026

An Abstract should not include more than 150 words.

Table of Contents

List of Figures	iv
List of Tables	v
Abbreviations	vi
1 Introduction	1
2 Literature Review	2
2.1 State-of-the-Art Solvers for Large-Scale CVRP	2
2.2 Scalability Strategies	2
2.2.1 Parallelization	2
2.2.2 Set Covering and Set Partitioning	2
2.2.3 Decomposition	3
2.2.4 Machine Learning	3
2.3 Algorithmic Frameworks	3
2.4 Remaining Research Gap	3
3 Problem Statement	4
4 Framework	5
4.1 Hierarchical Adaptive Operator Selection (HAOS)	5
4.2 Decomposition	8
4.3 Routing	10
4.4 Boundary-Guided Adaptive Iterated Local Search	11
4.5 Set Partitioning	11
4.6 Improvement	14
4.7 Pipeline Orchestration	15
5 Computational Study	16
5.1 Experimental Design	16
5.1.1 Framework Implementation	16
5.1.2 Dataset	16
5.1.3 Repititions	16
5.1.4 Hyperparameters	16
5.2 Results	17
5.2.1 Performance Comparison against State-of-the-Art Monolithic Solvers	17

5.2.2	Performance by Instance Characteristics	17
5.2.3	HAOS Weights	17
5.2.4	Convergence Behavior	17
5.2.5	Optional: Long-Run Study	17
5.2.6	Optional: Solability Study	18
6	Conclusion	19
6.1	Highlights	19
6.2	Limitations	19
6.3	Future Research	19
	Bibliography	20
	Appendix	24
1	Set Covering Formulation	24
2	Appendix Two	25

List of Figures

List of Tables

4.1 Default immediate and deferred HAOS rewards.	7
4.2 Inputs used by each decomposition method.	9

Abbreviations

CVRP Capacitated Vehicle Routing Problem.

1 Introduction

Motivation: Whats the real world problem and application, why is it important? Why are we doing this research now? Mention the BKS challenge and the project study as underlying motivators for this thesis.

Status Quo: What is the current state of the art? What are the limitations of the current state of the art? What are the challenges? Mention np-hardness and the issues that come with monolithic approaches at scale.

Research Questions: What is the focus of this thesis? Name primary and secondary research question and outline how they will be answered/evaluated.

Contributions: Specify the concrete contributions made by this thesis. This should include the general approach, and specifically the novel contributions of HAOS, BG-AILS, and the parallelized DRISPI framework approach.

Structure: Overview of the chapters and their content.

2 Literature Review

Cite papers with `\cite{BibTeX key}`

There is a comprehensive introduction to Operations Research by Winston (2007). It may be beneficial to consult comprehensive review articles within the relevant research domains (Dekker et al., 2012; Matl et al., 2018; Zhang and van Woensel, 2023).

Also use the introduction to this chapter to outline the general field of VRP and related problems. Explain np-hardness, the background VRP and CVRP, and how research has been progressing in the field.

2.1 State-of-the-Art Solvers for Large-Scale CVRP

Namedrop the big monolithic solvers for CVRP and their limitations. Especially mention the four solvers which are used in DRISPI: AILS-II, FILO, FILO2, and HGS-CVRP.

The main limitation to be mentioned here is the computational complexity and how it scales superexponentially with size of the problem.

Also explain the general approaches of these monoliths and what else is out there.

2.2 Scalability Strategies

Write here about different approaches to scale when it comes VRPs. This should include parallelization (-> prepares for DRISPI), decomposition or divide and conquer approaches (-> prepares for DRI), and maybe some recent machine learning approaches (-> prepares for HAOS).

2.2.1 Parallelization

xyz

2.2.2 Set Covering and Set Partitioning

xyz

2.2.3 Decomposition

xyz

2.2.4 Machine Learning

xyz

2.3 Algorithmic Frameworks

Idk about this section, but this would be a great way to introduce the approaches of Christoph (DRI for CCRPTW and DRSCI for HVRP). Maybe there are other approaches like this which include different strategies.

I need to be clear here what the difference between a framework, solver, and heuristic is. But if done well, this is a very nice way to introduce these terms and concepts and a good bridge to the research gap.

2.4 Remaining Research Gap

This should seamlessly connect to the research questions and pick them back up. It should present the state of the art approaches as unfit for the future of larger and larger VRPs.

Also, it should present that a combination of different strategies to tackle scale is unexplored and that the release of this new problem set is the perfect opportunity to explore this.

This section should get a sick figure/table to illustrate the research gap and how DRISPI combines all these scalability strategies which have been explored only in isolation so far.

3 Problem Statement

As introduced in Chapter 1, this work addresses the Capacitated Vehicle Routing Problem (CVRP) at a very-large scale. We now state the problem formally, discuss its computational complexity, and define the scope of the computational study.

The CVRP is defined on a complete undirected graph $G = (V, E)$, where $V = \{1\} \cup C$ is the set of locations. Node 1 represents the depot, and $C = \{2, \dots, n+1\}$ is the set of customers to be served, so that $n = |C|$. Each edge $\{i, j\} \in E$ carries a non-negative travel cost $d_{ij} \geq 0$, and each customer $i \in C$ has an associated demand $q_i > 0$. A homogeneous fleet of vehicles, each with a maximum capacity Q , is dispatched from and returns to the depot. The objective is to find a set of routes, each starting and ending at the depot, such that every customer is visited exactly once, no route exceeds the vehicle capacity, and the total travel cost is minimized.

The CVRP is NP-hard (Lenstra and Kan, 1981), a direct consequence of its generalization of the Traveling Salesman Problem. In practical terms, the number of feasible solutions grows factorially with n , rendering exact methods intractable beyond a few hundred customers. While branch-cut-and-price algorithms can solve small and medium instances to proven optimality (?), they are not viable at the scales considered here. This necessitates heuristic and metaheuristic approaches, which trade optimality guarantees for high-quality solutions within practical time budgets. Even within the heuristic paradigm, as Chapter 2 discusses, the cost of operating on the full problem simultaneously grows rapidly with n : neighborhood evaluation, memory requirements, and local search effectiveness all deteriorate at very-large scale, motivating the decomposition-based approach pursued in this work.

This work focuses specifically on the XL instances of the CVRPLib benchmark set (Queiroga et al., 2026), containing between 1,000 and 10,000 customers. These instances span a wide range of structural characteristics, including depot positioning, customer distribution, demand heterogeneity, and average route length, making them a demanding and representative testing set for large-scale CVRP solvers. DRISPI, proposed in Chapter 4, is designed specifically to operate at this scale by decomposing the full instance into tractable subproblems, each solved independently using existing state-of-the-art solvers.

4 Framework

General introduction to the framework. Build a figure to show how the framework would work sequentially here maybe (since this is easier to understand). It is just a short introduction.

The first part should also drop the name DRISPI for the first time and explain its components in one paragraph.

Pseudocode and a figure of the finally used framework architecture should come at the end of this chapter, since it won't be understandable at this point.

Instead, explain the main ideas behind the framework and namedrop important aspects for the later chapters (such as the route pool).

Maybe some of the following sections deserve subsection, but I am finding it quite clean to have it all on the same hierarchy level.

4.1 Hierarchical Adaptive Operator Selection (HAOS)

A heuristic configured for one routing instance can exploit that instance's size, spatial and demand distributions, and fleet capacity. Recent benchmark sets deliberately combine a wide range of these characteristics to test whether performance transfers across instance structures (Uchoa et al., 2017; Queiroga et al., 2026). Selecting a separate configuration for every instance would require a costly tuning campaign, while one fixed configuration must compromise across the benchmark set. HAOS adapts DRISPI's configuration during a run instead.

DRISPI repeatedly executes a decompose-route-improve (DRI) loop. An iteration first partitions the customers into smaller subclusters, solves one routing subproblem per subcluster, and concatenates the resulting routes. A boundary-guided improvement stage, BG-AILS, then revises routes near the borders between subclusters. All routes generated by these stages enter a persistent route pool. At scheduled intervals, a set covering or set partitioning (SC/SP) model recombines routes from this pool into a full solution, which receives a final unrestricted improvement pass. HAOS controls the decomposition and subsolver choices in the DRI loop, while the route pool lets later recombinations provide feedback about choices made in earlier iterations.

The principal methodological inspiration is adaptive large neighborhood search. Competing destroy and repair operators receive weights based on observed performance and are selected by roulette wheel (Ropke and Pisinger, 2006). The same adaptive layer has been applied across five vehicle routing variants (Pisinger and Ropke, 2007). HAOS adopts this principle but moves the adaptive choice outside a single local search: it controls decomposition granularity, the representation and method used for clustering, the demand contribution to the dissimilarity metric, and the subsolver. The adaptation is online because rewards update the weights while the current instance is being solved. The weights summarize observed performance on that instance; convergence to a globally optimal configuration is neither required nor guaranteed.

The options can also be interpreted as arms of a multi-armed bandit. After an arm is drawn, the search observes a reward but does not know what the unselected arms would have produced. This analogy connects HAOS to the exploration–exploitation problem (Auer et al., 2002) and to bandit-based adaptive operator selection (Fialho et al., 2010). HAOS does not implement a confidence-bound policy, an explicit exploration bonus, or a separate exploratory action. Its roulette probabilities follow the current weights directly. Stochastic sampling leaves every positive-weight option selectable, while the probability-floor transformation described below raises normalized shares that become sufficiently small.

Every DRI iteration draws one configuration across five levels. Level one selects the number of subclusters k from a scale-adaptive domain. Its base values are clipped to the instance’s minimum feasible fleet size and extended geometrically for larger instances, with at most 10 candidate values. Level two selects the demand weight λ_q from six evenly spaced values between zero and one; this value controls how strongly differences in customer demand influence the decomposition. Level three selects whether customers are clustered directly (the vertex paradigm) or through the routes of the current incumbent solution (the route paradigm). Level four selects a clustering method from the wheel associated with the chosen paradigm, with five vertex-based and three route-based methods in the deployed configuration. This conditional fourth level makes the selection hierarchical. Level five selects one of four routing solvers and applies it to every subcluster generated during the iteration. The first four levels therefore configure decomposition, and the fifth configures subproblem routing.

Two feasibility adjustments apply before the configuration is used. Route-based clustering requires an incumbent solution. Without one, HAOS uses the vertex paradigm and redraws the method from the vertex wheel; these applied choices receive the subsequent credit. If k exceeds the number of incumbent routes under route-based clustering, the applied k is clipped to the route count. The immediate update retains the index of the k arm originally drawn, while route tags record the configuration that was applied. Each iteration is logged with the product of

the five selected probabilities, which reports the joint probability of the realized configuration under the current wheels.

HAOS maintains six roulette wheels for the five levels, because the conditional method level has separate vertex and route wheels. Each wheel tracks one weight per option. A reward r adds to the weight of every arm selected for the credited configuration. Once per iteration, every weight is also multiplied by a common decay factor. Both operations retain the starting weight as a lower bound:

$$w_i \leftarrow \max(w_i + r, w_0), \quad (4.1)$$

$$w_i \leftarrow \max(\gamma w_i, w_0), \quad \gamma = 0.95 \text{ by default}, \quad (4.2)$$

with every wheel starting from the same weight $w_0 = 10.0$. No separate recency score is computed; uniform decay is the only temporal mechanism in the weights. Before an arm is drawn, HAOS first computes its normalized weight $\hat{p}_i = w_i / \sum_l w_l$. A level-specific floor is applied to these values, followed by renormalization:

$$p_i = \frac{\max(\hat{p}_i, p_{\min})}{\sum_j \max(\hat{p}_j, p_{\min})}. \quad (4.3)$$

The floor parameter is 0.025 for k , 0.05 for λ_q , the clustering methods, and the solver, and 0.10 for the paradigm. Because Equation 4.3 renormalizes after flooring, these values are nominal pre-normalization floors rather than strict lower bounds on the final probabilities. HAOS retains three elements of the adaptive layer proposed by Ropke and Pisinger (2006) and used for general vehicle routing by Pisinger and Ropke (2007): initially equal weights, quality-dependent scores, and roulette selection proportional to the resulting weights. Its additive reward, global decay, and six-wheel hierarchy are specific to DRISPI.

HAOS has an immediate channel for results produced directly by a DRI iteration and a deferred channel for routes that prove useful during later pool recombination. Table 4.1 gives the deployed reward values. Both channels compare a candidate with the incumbent best cost and the previous iteration's cost, but the deferred scores are smaller because several earlier configurations can contribute routes to the same recombined solution.

Table 4.1: Default immediate and deferred HAOS rewards.

Outcome	Immediate	Deferred
New incumbent best	10.0	7.0
Improvement over previous iteration only	4.0	3.0
No improvement	1.0	0.0
No solution	0.0	0.0

The immediate channel scores two candidates: the solution obtained after decom-

position and subproblem routing, and the solution obtained after BG-AILS. Both rewards are credited to the same five-level configuration drawn for that iteration and therefore accumulate. Each selected arm receives the full reward. HAOS learns from the performance of the complete configuration but does not isolate the contribution of one level or model interactions between levels.

The deferred channel is evaluated after SC/SP has selected routes from the pool and the resulting solution has received its unrestricted improvement pass. Every stored route carries a HAOS tag containing the configuration under which it was generated. The deferred reward is credited once to every distinct tag among the routes selected by SC/SP, even if those tags originate from much earlier iterations. Routes created only during the unrestricted improvement pass are excluded because no HAOS selection produced them. Deferred attribution is necessary because the usefulness of one stored route may become visible only when it is combined with routes generated later.

During the first 10 iterations, HAOS performs the same five-level draw but suppresses both reward updates and decay. All weights therefore remain equal, making every option uniform within the wheel from which it is eligible to be drawn. This warm-up lets the first incumbent and an initial route pool form before early outcomes influence later probabilities. Learning begins with iteration 10, after which the immediate and deferred updates follow the same weight system.

4.2 Decomposition

The decomposition layer runs at the beginning of every iteration in the DRI loop, adapting the decompose-route-improve terminology of Kerscher and Minner (2025), who propose the framework for large-scale vehicle routing problems with time windows that this work extends to the CVRP at very-large scale in a fully parallelized, self-tuning setting. A related line of work pairs the same customer- and route-based decomposition with set covering, rather than DRISPI’s SC/SP model (Section 4.5), to solve large-scale heterogeneous-fleet routing problems (Kerscher et al., 2025). HAOS (Section 4.1) controls this stage through four levers, redrawn at every iteration: first the number of subclusters k ; second the clustering paradigm, either vertex-based, which partitions the customers directly, or route-based, which partitions the routes of the current incumbent solution and propagates the resulting labels to their customers; third the clustering method used within that paradigm; and fourth the demand weight λ_q of the dissimilarity metric utilized in the majority of clustering approaches. A fifth quantity that also feeds the metric, the angular offset ϕ , is drawn uniformly from $[0, 2\pi)$ at every iteration independently of HAOS, so that the wraparound point described below does not stay fixed across iterations.

The dissimilarity metric combines three terms for every pair of customers i and j with coordinates (x_i, y_i) and (x_j, y_j) and demands q_i and q_j : their Euclidean dis-

tance, an angular term, and a demand term,

$$d_{ij} = \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2 + \lambda_\theta \Delta\theta_{ij}^2} \left(1 + \lambda_q \frac{|q_i - q_j|}{Q} \right). \quad (4.4)$$

$\Delta\theta_{ij}$ is the shortest angular distance between customer i 's and customer j 's polar angle around the depot, computed with the offset ϕ introduced above. λ_θ scales the angular term to the average coordinate magnitude of the instance, so that neither term dominates across instances of different spatial extent, and ϕ shifts where the $\Delta\theta_{ij}$ wraparound at 2π falls. The demand factor applies the multi-feature dissimilarity metric of Kerscher and Minner (2025), who combine spatial, temporal, and demand features, to a setting without time windows: only the demand ratio $\lambda_q |q_i - q_j|/Q$ remains alongside space.

Table 4.2: Inputs used by each decomposition method.

Method	Paradigm	Input	Angular	Demand
K-Means	vertex	raw coordinates	—	—
Fuzzy C-Means	vertex	raw coordinates	—	—
K-Medoids	vertex	dissimilarity matrix	✓	✓
Agglomerative (Average)	vertex	dissimilarity matrix	✓	✓
Agglomerative (Complete)	vertex	dissimilarity matrix	✓	✓
K-Means	route	route centroids	—	—
Agglomerative (Average)	route	route centroids	—	—
Agglomerative (Complete)	route	route centroids	—	—

Both clustering paradigms trace back to the classic cluster-first, route-second scheme for capacitated routing (Gillett and Miller, 1974; Fisher and Jaikumar, 1981). Vertex-based clustering partitions the customers directly and offers five methods (Table 4.2). K-Means (MacQueen, 1967) and Fuzzy C-Means (Bezdek et al., 1984) cluster on raw two-dimensional coordinates, Fuzzy C-Means hardened to a partition by assigning each customer to its highest-membership cluster. K-Medoids, i.e., partitioning around medoids (Kaufman and Rousseeuw, 1990), and Agglomerative hierarchical clustering under average or complete linkage (Kaufman and Rousseeuw, 1990) instead cluster directly on the dissimilarity matrix of Equation 4.4, the only three method arms, out of eight, that λ_q and ϕ reach.

Route-based clustering represents each route by the centroid of its customers' coordinates and clusters the centroids with K-Means or with the same average or complete linkage, applied to the plain Euclidean distance between centroids rather than to the dissimilarity matrix (Table 4.2), echoing the barycenter-clustering decomposition strategy studied by Santini et al. (2023). The matrix, rebuilt from (λ_q, ϕ) , still feeds the boundary detection used in BG-AILS (Section 4.4) regardless of which paradigm decomposition uses. Labels then propagate from routes to customers di-

rectly, since every customer belongs to exactly one route in the incumbent solution; a nearest-centroid tie-break in the implementation matters only for the degenerate case of conflicting cluster candidates.

The clusters are then handed over in form of a set of independent subproblems to the routing layer (Section 4.3) to be solved.

4.3 Routing

Once decomposition produces the k subclusters (Section 4.2), the routing stage builds one sub-instance per subcluster, with contiguous internal node IDs and its own copy of the depot, and solves all k of them in parallel, independently of each other. A fifth HAOS lever, the solver, is drawn once per iteration and applied to every subcluster that iteration, so all k sub-instances are solved by the same one of four solvers. Each sub-instance receives a time budget of $\max(\text{floor}, \text{size} \times \text{rate})$ seconds, with a default rate of 0.06 seconds per customer and a floor of 5 seconds, so small subclusters are not starved of search time. The k sub-instances are dispatched to a pool of worker processes pinned to the cores reserved for this stage; if k exceeds the worker count, the remainder queue in further waves, and Section 4.7 details how this pool fits into the wider core split. A wall-clock timeout derived from a fitted wave-time model guards against a hung worker.

The four solvers are monolithic, full-instance CVRP heuristics that DRISPI treats as black boxes: AILS-II (Máximo et al., 2024)¹, an adaptive iterated local search with an elite-solution pool for large instances that extends an earlier adaptive ILS hybridized with path-relinking (Máximo and Nascimento, 2021), run as a Java subprocess; FILO (Accorsi and Vigo, 2021)², a ruin-and-recreate local search built on the COBRA granular-neighborhood library, run as a compiled C++ subprocess; FILO2 (Accorsi and Vigo, 2024)³, an evolution of FILO for instances up to one million customers, also a C++ subprocess; and PyVRP (Wouda et al., 2024)⁴, a Python package implementing hybrid genetic search (Vidal et al., 2012; Vidal, 2022) that DRISPI calls in-process rather than as a subprocess. These four are not an arbitrary shortlist: Queiroga et al. (2026), who introduce the XL instance set this thesis targets (Chapter 3), benchmark them against several other state-of-the-art heuristics and report that AILS-II and FILO2 dominate the best-known solutions on the XL range itself, while HGS-CVRP remains the strongest performer on the smaller, medium-scale instances that a typical DRISPI subcluster is closer in size to; the solver lever spans exactly this size spectrum.

The routes returned by the k solved subclusters are concatenated into one combined solution for the full instance; because the subclusters partition the customers,

¹<https://github.com/INFORMSJoC/2023.0106>

²<https://github.com/acco93/filo>

³<https://github.com/acco93/filo2>

⁴<https://pyvrp.org> (pip install pyvrp).

no route returned at this stage spans two subclusters, and the concatenation needs no reconciliation. Each route is tagged with the subcluster of its customers, information that BG-AILS (Section 4.4) reuses once its perturbations start moving customers across subcluster boundaries. This combined, subcluster-tagged solution is the input BG-AILS perturbs and re-optimizes next.

4.4 Boundary-Guided Adaptive Iterated Local Search

Explain the logic of BG AILS. Show the important parameters and some nice plots to illustrate the approach.

I want one plot to explain route selection and perturbation.

And I want one plot that shows the difference between BG-AILS and AILS. (Highlight the nodes that were touched, intensify color by number of touches)

Since this is one of my novelties, it can be a longer section.

4.5 Set Partitioning

Periodically, DRISPI steps back from the current incumbent and recombines every route accumulated so far into one new candidate solution, using set covering or set partitioning (SC/SP) as the recombination model. Alternating a metaheuristic with an exact set-partitioning solve in this way is an established matheuristic pattern (Jourdan et al., 2009); Cavaliere et al. (2022) apply essentially this pairing to large-scale CVRP instances directly, alternating LKH-3 local search with a restricted set-partitioning solve. Doing so only pays off if a shared memory of routes exists for it to recombine, so this section first describes that memory, the route pool, before turning to the SP and SC models themselves and the hyperparameter that decides between them.

The route pool stores one entry per distinct customer set $C_r \subseteq C$, where C is the customer set of the instance (Chapter 3), keyed by the frozen customer set rather than by the ordered sequence of customers a route visits them in: two routes serving the same customers in a different order, or at a different cost, are the same column, and the cheaper of the two overwrites the other. This column-oriented memory, populated online during the search and periodically recombined through a partitioning model, follows the adaptive-memory principle of Rochat and Taillard (1995), who introduce the route pool and its set-partitioning recombination for vehicle routing.

Two pathways feed the pool every iteration, both through the same contribute-before-adopt rule: a candidate solution is added to the pool before the pipeline

decides whether to adopt it as the new incumbent, so a losing candidate still leaves its routes behind for later reuse. The DRI loop feeds the pool twice per iteration, once from its decompose-route stage (Section 4.3) and once from its improve stage, BG-AILS (Section 4.4). The SC/SP step covered in this section feeds the pool only indirectly: solving SC or SP selects among the customer sets already in the pool and creates no new ones, so its own output adds nothing; the standard-AILS pass that polishes its result (Section 4.6) is what can move customers between routes and thereby contribute genuinely new columns. Every contributed route carries a tag recording the full HAOS selection that produced it, that is, k , the demand weight, the paradigm, the method, and the solver, with a separate flag marking routes from the post-SC/SP polish; the tag survives even once HAOS has moved on to a different selection, which is what later lets reward attribution and pool inspection trace a route back to its origin. Whenever a candidate is adopted as the new incumbent, its routes, already in the pool by the rule above, are marked elite; elite routes are exempt from eviction, so the pool never loses the one partition it is guaranteed to already have, namely the incumbent's own.

Every SC/SP solve also updates two running scores on each pool entry. The quality score is the continuous weight the LP relaxation of that solve (below) assigns to the entry's route, appended to a window of the three most recent values; an entry never touched by a solve carries no quality score yet. The diversity score is the mean Jaccard distance (Jaccard, 1901) between a route's customer set and every other pool entry's customer set,

$$\bar{J}(r) = \frac{1}{|R| - 1} \sum_{r' \in R \setminus \{r\}} \frac{|C_r \Delta C_{r'}|}{|C_r \cup C_{r'}|}, \quad (4.5)$$

where R is the pool at the time of the update, recomputed for every entry whenever any solve happens and likewise appended to a three-value window. Both scores are read as the mean of their window, so a single unusually high or low value does not dominate a route's standing.

Eviction ranks the non-elite entries on both scores and combines the ranks into one fitness value. Among the entries that have been scored at least once, ascending order by mean quality score gives rank one to the weakest route and the highest rank to the strongest; independently, descending order by mean diversity score gives rank one to the most singular route and the highest rank to the most redundant. Writing $\rho_q(r)$ and $\rho_d(r)$ for these two ranks, fitness is $\Phi(r) = \rho_q(r) + w_d \cdot \rho_d(r)$, with the diversity weight w_d (0.5 by default) controlling how much redundancy counts against a route relative to raw quality; the entries with the highest $\Phi(r)$ are evicted first. Because both terms grow toward the same end of their respective orderings, a route that the LP relies on heavily but that is also nearly identical to several other pool entries can rank as a stronger eviction candidate than a weak but singular one, since redundancy drives the diversity term up regardless of quality. Eviction only

triggers once the pool exceeds its maximum size (10,000 columns by default) and only removes the overflow amount; entries never yet scored are taken only if the overflow cannot be covered by scored, non-elite entries alone, and elite entries are never taken at all.

The recombination model itself treats the pool as a column set. Let R be the set of routes currently in the pool, c_r the cost of route $r \in R$, that is, the Euclidean depot-customers-depot round trip over the travel costs of Chapter 3, and $x_r \in \{0, 1\}$ a decision variable selecting route r . Set partitioning, in the formulation Balinski and Quandt (1964) introduce for exactly this delivery-routing setting, reads

$$\min \sum_{r \in R} c_r x_r \quad (4.6)$$

$$\text{s.t.} \quad \sum_{r \in R: i \in C_r} x_r = 1, \quad \forall i \in C \quad (4.7)$$

$$x_r \in \{0, 1\}, \quad \forall r \in R. \quad (4.8)$$

Solving Equations 4.6–4.8 recombines route fragments that were never jointly considered, since they may have entered the pool from different subclusters, different HAOS selections, or different iterations, into one coherent solution that still covers every customer exactly once. This is the sense in which SP looks across the boundaries any single DRI iteration’s decomposition imposes, something no individual subcluster solve or BG-AILS perturbation can do on its own.

DRISPI builds this model once in Gurobi⁵ and solves it twice. It first relaxes Equation 4.8 to the continuous $x_r \in [0, 1]$ and solves the resulting linear program; the fractional weights this LP relaxation returns are exactly the quality scores fed back into the route pool above, so every SC/SP solve doubles as a scoring pass over the pool regardless of what it ultimately selects, in the same spirit in which column-generation approaches read pricing information out of LP duals (Lübbecke and Desrosiers, 2005), though DRISPI’s pool is populated by heuristic solves rather than a pricing subproblem. The variables are then switched to binary, warm-started from the LP solution, and re-optimized as a mixed-integer program under a wall-clock time limit and a relative optimality gap (720 seconds and 0.025% by default), so the search can be cut off with a good, if not provably optimal, solution rather than run to completion.

The equality constraint of Equation 4.7 is only a meaningful choice when the pool offers more than one real alternative for most customers; a pool still dominated by a single incumbent partition would just reproduce it under SP. DRISPI therefore checks, before every SC/SP trigger, whether every customer in the instance is covered by at least a fixed minimum number of pool routes, five by default. Only when that holds does SP run; otherwise DRISPI falls back to set covering (SC), the inequality-constrained relaxation of the same formulation (Garfinkel and

⁵<https://www.gurobi.com>

Nemhauser, 1969), whose formal problem statement and LP relaxation are given in Appendix 1. SC only requires every customer to be covered at least once rather than exactly once, a condition the pool always satisfies, because the elite routes described above are never evicted and already form a feasible partition on their own. This is also why the fallback is safe and not merely convenient: SP is never blocked by infeasibility, only by a pool that has not yet accumulated enough alternatives, and SC exists to give the SC/SP step something well defined and always solvable to run on every scheduled trigger, warmup aside, no matter how far the pool has matured. The price of that permissiveness is that SC's selected routes can overlap on a customer; a savings-based heuristic then removes each duplicated customer from every selected route except the one where removing it would save the least cost, restoring a partition before the standard-AILS pass (Section 4.6) polishes the result.

4.6 Improvement

Once SC/SP (Section 4.5) recombines the route pool into one partition, DRISPI runs one further pass over it using AILS-II as solver, seeded only with that partition as an initial solution. Other than in BG-AILS (Section 4.4), this step focuses less on perturbation but rather refinement by local search. This standard improvement step also allows the solver to touch all routes instead of biasing moves towards certain neighborhoods. The pass runs under its own wall-clock budget (360 seconds by default) on the same core reservation as the SC/SP solve that precedes it; Section 4.7 covers how exactly a parallel architecture of DRISPI looks.

The refined routes feed back into the route pool exactly like every other stage's output (see Section 4.5): a route whose customer set matches one SC/SP already selected keeps that route's existing tag, while a route AILS-II creates by moving customers between routes is added as new, tagged as an improvement route rather than credited to a HAOS selection of its own.

This is also where the deferred half of HAOS's reward system (Section 4.1) closes. The pass's outcome, a new best, an improvement over the previous iteration without beating the incumbent, or neither, is scored through the deferred branch of HAOS's reward function and credited not to the current iteration's own draw but to every distinct HAOS tag the routes SC/SP selected already carried, that is, to whichever earlier decompose-route or BG-AILS draws produced those specific routes, regardless of how many iterations ago that was. Improvement-route tags are excluded from this credit, since AILS-II created them without any HAOS draw of their own to reward.

4.7 Pipeline Orchestration

This section should lead all previous sections together.

Here I need to explain with a figure how the pipeline is parallelized (the 6+1+1 approach) to minimize idle cores.

I also need to write a long pseudocode explaining the entire pipeline logic. This could be hard if done in the parallelized setting. Will decide how to do this later.

5 Computational Study

xyz?

5.1 Experimental Design

xyz?

5.1.1 Framework Implementation

Write which hardware is used and which external components/implementations are used in the utilized framework (Solvers, Gurobi, etc.).

5.1.2 Dataset

Mention the new XL dataset, how it is generated and composed (the different levers n , k , dep , customer distribution, demand distribution, Q , r).

Also explain how they were initially evaluated and what existing solutions are used for the later benchmarking (AILS-II, FILO, FILO2, and HGS-CVRP, maybe DRSCI). Explain how current BKS were found through the BKS challenge and that they serve as the baseline instead of the numbers from the original paper. (Provide link and date to the official website from which the BKS values were obtained before the final benchmark).

5.1.3 Repititions

Explain the number of seeds used and why. Reference Law and Kelton here and show the statistics behind why this is clean and answers the research question properly.

Name of the chapter is object to change.

5.1.4 Hyperparameters

Show the hyperparameters used for the benchmark run. Explain the impact of the different hyperparameters on the performance.

5.2 Results

Short introduction.

5.2.1 Performance Comparison against State-of-the-Art Monolithic Solvers

Show the results of the 100 runs a 2h with 3 seeds each. Make a big table with the avg. and best results for each instance. Next to it take the values from the Queiroga et al. (2026) paper and show the difference.

This answers the primary research question.

5.2.2 Performance by Instance Characteristics

Show how DRISPI performs by instance characteristics. This should be a simple plot showing the performance by instance size, depot position, customer distribution, and demand distribution.

5.2.3 HAOS Weights

Investigate how HAOS weights are converging and if there are obvious tendencies. Interpret the results.

Are some HAOS levels converging always to the same result? Are some HAOS levels always converging in a similar way subject to the instance characteristics? Whats the variance in HAOS weight convergence? Do they differ a lot from seed to seed and are sensitive to randomness?

Are the results interpretable in general?

5.2.4 Convergence Behavior

Show how the convergence behavior is like for DRISPI by wallclock time and iterations. Use this chapter to also investigate improvement sources (quantitatively and qualitatively).

5.2.5 Optional: Long-Run Study

Another study to show the performance of DRISPI with more time budget. This should focus especially on instances with solutions close to the BKS.

5.2.6 Optional: Scalability Study

Another study to show the diminishing returns of the scalability through parallelization. Use the same approach as in the scalability chapter of the literature.

6 Conclusion

Short introduction. It should relect on the primary and secondary research questions.

6.1 Highlights

Mention the strengths (such as adaptiveness through online learning) and weaknesses (such as more CPU time required) of the framework.

Also elaborate on the main contributions of the thesis.

6.2 Limitations

Lay out the limitations of the study. It should include the difficulty of comparing single-threaded monolithic solvers to the parallelized framework (CPU time vs. wallclock time, different hardware, etc.).

Also mention limitations and variations during the run when it comes to harware and number of seeds. Computation was a limiting factor here and definitely influenced performance.

6.3 Future Research

Use the HAOS weights to train an offline learning model and inject the results back into the framework. This effectively eliminates the warm-up period and will lead to a substantial speed-up of the framework.

Also mention that the framework, other than monolithic solvers, is highlz adaptable to different instances and VRP variants. Since it only requires the plug-and-play of capable clustering methods andsolvers on the their respective stages.

Bibliography

- Accorsi, L., D. Vigo. 2021. A fast and scalable heuristic for the solution of large-scale capacitated vehicle routing problems. *Transportation Science*, 55 (4), 832-856, doi: 10.1287/trsc.2021.1059.
- Accorsi, L., D. Vigo. 2024. Routing one million customers in a handful of minutes. *Computers & Operations Research*, 164 106562, doi: 10.1016/j.cor.2024.106562.
- Auer, P., N. Cesa-Bianchi, P. Fischer. 2002. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47 (2-3), 235-256, doi: 10.1023/A:1013689704352.
- Balinski, M. L., R. E. Quandt. 1964. On an integer program for a delivery problem. *Operations Research*, 12 (2), 300-304, doi: 10.1287/opre.12.2.300.
- Bezdek, J. C., R. Ehrlich, W. Full. 1984. FCM: The fuzzy c-means clustering algorithm. *Computers & Geosciences*, 10 (2-3), 191-203, doi: 10.1016/0098-3004(84)90020-7.
- Cavaliere, F., E. Bendotti, M. Fischetti. 2022. An integrated local-search/set-partitioning refinement heuristic for the Capacitated Vehicle Routing Problem. *Mathematical Programming Computation*, 14 (4), 749-779, doi: 10.1007/s12532-022-00224-2.
- Dekker, R., J. Bloemhof, I. Mallidis. 2012. Operations research for green logistics – an overview of aspects, issues, contributions and challenges. *European Journal of Operational Research*, 219 (3), 671-679, doi: 10.1016/j.ejor.2011.11.010.
- Fialho, Á., L. Da Costa, M. Schoenauer, M. Sebag. 2010. Analyzing bandit-based adaptive operator selection mechanisms. *Annals of Mathematics and Artificial Intelligence*, 60 (1-2), 25-64, doi: 10.1007/s10472-010-9213-y.
- Fisher, M. L., R. Jaikumar. 1981. A generalized assignment heuristic for vehicle routing. *Networks*, 11 (2), 109-124, doi: 10.1002/net.3230110205.
- Garfinkel, R. S., G. L. Nemhauser. 1969. The set-partitioning problem: Set covering with equality constraints. *Operations Research*, 17 (5), 848-856, doi: 10.1287/opre.17.5.848.
- Gillett, B. E., L. R. Miller. 1974. A heuristic algorithm for the vehicle-dispatch problem. *Operations Research*, 22 (2), 340-349, doi: 10.1287/opre.22.2.340.

- Jaccard, P. 1901. Distribution de la flore alpine dans le bassin des dranses et dans quelques régions voisines. *Bulletin de la Société Vaudoise des Sciences Naturelles*, 37 241-272, doi: 10.5169/seals-266440.
- Jourdan, L., M. Basseur, E.-G. Talbi. 2009. Hybridizing exact methods and metaheuristics: A taxonomy. *European Journal of Operational Research*, doi: 10.1016/J.EJOR.2007.07.035.
- Kaufman, L., P. J. Rousseeuw. 1990. *Finding Groups in Data: An Introduction to Cluster Analysis*. Wiley Series in Probability and Statistics, John Wiley & Sons, New York, doi: 10.1002/9780470316801.
- Kerscher, C., S. Minner. 2025. Decompose-route-improve framework for solving large-scale vehicle routing problems with time windows. *Transportation Research Part E: Logistics and Transportation Review*, 204 104409, doi: 10.1016/j.tre.2025.104409.
- Kerscher, C., S. Minner, F. Lehuédé, G. Massonnet. 2025. Solving Heterogeneous Vehicle Routing Problems Using Decomposition and Set Covering, .
- Lenstra, J. K., A. H. G. R. Kan. 1981. Complexity of vehicle routing and scheduling problems. *Networks*, 11 (2), 221-227, doi: 10.1002/net.3230110211.
- Lübbecke, M. E., J. Desrosiers. 2005. Selected Topics in Column Generation. *Operations Research*, 53 (6), 1007-1023, doi: 10.1287/opre.1050.0234.
- MacQueen, J. B. 1967. Some methods for classification and analysis of multivariate observations. *Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability*, vol. 1. University of California Press, Berkeley, CA, 281-297. URL <https://projecteuclid.org/euclid.bsmsp/1200512992>.
- Matl, P., R. F. Hartl, T. Vidal. 2018. Workload equity in vehicle routing problems: A survey and analysis. *Transportation Science*, 52 (2), 239-260, doi: 10.1287/trsc.2017.0744.
- Máximo, V. R., J.-F. Cordeau, M. C. V. Nascimento. 2024. AILS-II: An adaptive iterated local search heuristic for the large-scale capacitated vehicle routing problem. *INFORMS Journal on Computing*, 36 (4), 974-986, doi: 10.1287/ijoc.2023.0106.
- Máximo, V. R., M. C. V. Nascimento. 2021. A hybrid adaptive iterated local search with diversification control to the capacitated vehicle routing problem. *European Journal of Operational Research*, 294 (3), 1108-1119, doi: 10.1016/j.ejor.2021.02.024.
- Pisinger, D., S. Ropke. 2007. A general heuristic for vehicle routing problems. *Computers & Operations Research*, 34 (8), 2403-2435, doi: 10.1016/j.cor.2005.09.012.

- Queiroga, E., R. Martinelli, A. Subramanian, E. Uchoa, T. Vidal. 2026. The XL instances and the CVRPLib best known solution challenge. doi: 10.48550/arXiv.2601.11467. ArXiv:2601.11467.
- Rochat, Y., E. D. Taillard. 1995. Probabilistic diversification and intensification in local search for vehicle routing. *Journal of Heuristics*, 1 (1), 147-167, doi: 10.1007/BF02430370.
- Ropke, S., D. Pisinger. 2006. An Adaptive Large Neighborhood Search Heuristic for the Pickup and Delivery Problem with Time Windows. *Transportation Science*, 40 (4), 455-472, doi: 10.1287/trsc.1050.0135.
- Santini, A., M. Schneider, T. Vidal, D. Vigo. 2023. Decomposition Strategies for Vehicle Routing Heuristics. *INFORMS Journal on Computing*, 35 (3), 543-559, doi: 10.1287/ijoc.2023.1288.
- Uchoa, E., D. Pecin, A. Pessoa, M. Poggi, T. Vidal, A. Subramanian. 2017. New benchmark instances for the Capacitated Vehicle Routing Problem. *European Journal of Operational Research*, 257 (3), 845-858, doi: 10.1016/j.ejor.2016.08.012.
- Vidal, T. 2022. Hybrid genetic search for the CVRP: Open-source implementation and SWAP* neighborhood. *Computers & Operations Research*, 140 105643, doi: 10.1016/j.cor.2021.105643.
- Vidal, T., T. G. Crainic, M. Gendreau, N. Lahrichi, W. Rei. 2012. A Hybrid Genetic Algorithm for Multidepot and Periodic Vehicle Routing Problems. *Operations Research*, 60 (3), 611-624, doi: 10.1287/opre.1120.1048.
- Winston, W. L. 2007. *Operations research: Applications and algorithms*. 4th ed. Brooks/Cole Thomson Learning, Belmont 2007.
- Wouda, N. A., L. Lan, W. Kool. 2024. PyVRP: A high-performance VRP solver package. *INFORMS Journal on Computing*, 36 (4), 943-955, doi: 10.1287/ijoc.2023.0055.
- Zhang, J., T. van Woensel. 2023. Dynamic vehicle routing with random requests: A literature review. *International Journal of Production Economics*, 256 108751, doi: 10.1016/j.ijpe.2022.108751.

Appendix

1 Set Covering Formulation

Section 4.5 introduces set covering (SC) as the fallback DRISPI runs in place of set partitioning (SP) when the route pool does not yet cover every customer at least a minimum number of times. With R the route pool at the time of the solve, c_r the cost of route $r \in R$, C the customer set of the instance (Chapter 3), $C_r \subseteq C$ the customer set of route r , and $x_r \in \{0, 1\}$ selecting route r , SC relaxes the partition equality of SP (Equation 4.7) to the cover inequality of Garfinkel and Nemhauser (1969), who characterize set partitioning as set covering under an equality constraint:

$$\min \sum_{r \in R} c_r x_r \quad (1)$$

$$\text{s.t.} \quad \sum_{r \in R: i \in C_r} x_r \geq 1, \quad \forall i \in C \quad (2)$$

$$x_r \in \{0, 1\}, \quad \forall r \in R. \quad (3)$$

As with SP, DRISPI solves the LP relaxation of Equation 3, $x_r \in [0, 1]$, first; the resulting fractional weights update the pool's quality scores in the same way regardless of whether the solve ends up running SC or SP, and the binary re-solve is warm-started from that relaxation.

2 Appendix Two

Declaration Of Authorship

I hereby solemnly declare that I have completed the present work independently. All ideas taken directly or indirectly from external sources have been clearly identified as such.

I am aware that the work may be examined in digital form to determine whether unauthorized aids were used and whether it constitutes - either in whole or in part - plagiarism. For the purpose of comparison with existing sources, my work may be entered into a database and, after review, may remain there for comparison with future submissions. No further reproduction or usage rights are thereby granted. This work has neither been submitted to another examination authority nor published.

Place, Date

Maximilian Mustermann

Declaration of AI Use

Download the Declaration of AI Use from the website, complete it truthfully, and include it as a separate page at the end of your final thesis PDF here instead of this page.